


INDIAN INSTITUTE OF TECHNOLOGY MADRAS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

---

# Course Project Report:

# Protect Privacy Through Forgetting: Machine Unlearning for Vision Models

---

CS6886: SYSTEMS ENGINEERING FOR DEEP LEARNING

**Submitted by:**

Saurabh Chatterjee
Roll No: NS26Z334

Reva Bharara
Roll No: NS26Z212

**GitHub Repository:**

https://github.com/chatterjeesaurabh/Machine-Unlearning-for-Vision-Models

November 2026

# Summary Sheet

| Item | Value |
|---|---|
| Task | Machine unlearning for image classification: remove the influence of a *forget set* from a trained model without retraining from scratch |
| Model / data | ResNet-18 on CIFAR-10 (45k train pool, 5k validation, 10k test) |
| Original model | 94.82% test accuracy (182 epochs, ≈7 min on one A100) |
| Forgetting scenarios | S1: a whole class (airplane, 4,500 images); S2: random 10/30/50% of three classes (cat, dog, truck) |
| Gold reference | Retrain from scratch on the retain set (12 references, 3 seeds; 27 more for entanglement) |
| Methods implemented | Gradient Ascent, GA + Descent, Teacher-Student, Random Relabeling, Boundary Relabeling; each ± Gradient based Weight Saliency |
| Evaluations | Forget/retain/test accuracy, loss-based MIA, avg_gap to Retrain, entanglement (2 experiments), linear probe / NCC / k-NN purity, perturbed forget samples, post-unlearning quantization, run time and GPU memory |
| Total recorded runs | 1,845 (every run: config, history, metrics, per-sample outputs) |
| Best class forgetting (S1) | Boundary Relabeling (our C1-mix): forget 0.00%, retain 100%, MIA 51.0% (Retrain 51.0%), avg_gap **0.16** (paper setting: 6.92) in 28 s vs 402 s retraining |
| Best subset forgetting (S2) | Random Relabeling + WS (S2-30: 3.10, S2-50: 2.58), GA + Descent (S2-10: 3.52); doing nothing: 4.5–5.3 |
| Key findings | BatchNorm-statistics drift fakes forgetting in forget-only methods (fixed with mixed BN batches); forgotten class stays linearly decodable (probe 82–94%); entangled samples are harder to forget in subset forgetting; INT8/6/4 quantization never restores forgotten knowledge |
| GitHub Repository | https://github.com/chatterjeesaurabh/Machine-Unlearning-for-Vision-Models |

**Environment.** Python 3.11, PyTorch 2.4.0 + torchvision 0.19, NumPy, scikit-learn, Matplotlib; NVIDIA A100-PCIE-40GB. Seeds 0, 1, 2 for every reported configuration (mean over seeds unless stated).

# Contents

**1 Introduction: Machine Unlearning** **3**
1.1 What is machine unlearning? . . . 3
1.2 Why is it needed? . . . 3
1.3 Applications in vision . . . 3
1.4 Forgetting scenarios and difficulty . . . 3
1.5 Approaches to unlearning . . . 4
1.6 Project objectives . . . 4

**2 Experimental Setup** **4**

**3 Unlearning Methods** **5**
3.1 M1: Gradient Ascent (GA) . . . 5
3.2 M2: Gradient Ascent + Descent (GA + GD) . . . 5
3.3 M3a: Teacher-Student unlearning . . . 5
3.4 M3b-R: Random Relabeling . . . 6
3.5 M3b-B: Boundary Relabeling . . . 6
3.6 M4: Gradient based Weight Saliency (WS) . . . 6

**4 Evaluation Metrics** **6**
4.1 Accuracy and distance to Retrain . . . 6
4.2 Membership Inference Attack (MIA) . . . 7
4.3 Embedding entanglement . . . 7
4.4 Embedding separability: linear probe, NCC, k-NN purity . . . 8
4.5 Robustness: perturbed forget samples and quantization . . . 8

**5 Results** **8**
5.1 Accuracy and MIA . . . 9
5.2 Embedding separability (linear probe) . . . 10
5.3 Entanglement and unlearning difficulty . . . 11
5.4 Effect of Gradient based Weight Saliency . . . 12
5.5 Robustness to perturbed forget samples . . . 13
5.6 Post-unlearning quantization . . . 13
5.7 Computational cost . . . 14

**6 Method-specific Observations and Improvements** **15**
6.1 The BatchNorm-statistics problem (Gradient Ascent, Boundary Relabeling, GA + Descent) . 15
6.2 Teacher-Student: soft-label re-teaching and collapse . . . 15
6.3 Boundary Relabeling: finding the nearest class . . . 16
6.4 Other issues found and fixed . . . 16

**7 Overall Comparison of the Methods** **16**

**8 Discussion: why subset forgetting looks "unsuccessful"** **17**

**9 Conclusion** **17**

# 1 Introduction: Machine Unlearning

## 1.1 What is machine unlearning?

A trained network stores information about its training data in its weights. When a part of the data must be removed after training, deleting the files is not enough: the model still behaves as if it had seen them. *Machine unlearning* removes the influence of a selected subset of the training data — the **forget set** $D_f$ — from an already trained model, while preserving its behaviour on the remaining data — the **retain set** $D_r = D \setminus D_f$.

The ideal result is the model obtained by **retraining from scratch on** $D_r$ (exact unlearning). Retraining is the gold standard but costs a full training run for every deletion request. *Approximate unlearning* instead starts from the original model $\theta_o$ and applies a short update $\theta_o \to \theta_u$ (a few epochs or less) so that $\theta_u$ behaves like the retrained model $\theta_r$. Its quality therefore has to be *measured* against $\theta_r$ (Fig. 1).

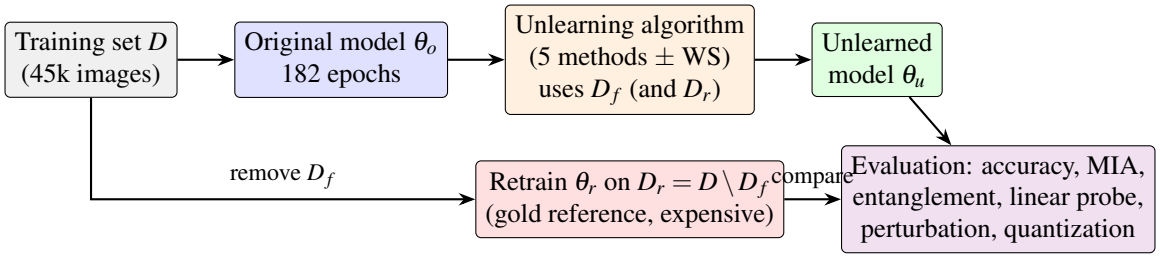



**Figure 1:** Approximate unlearning and its evaluation against the retrained-from-scratch reference.

## 1.2 Why is it needed?

- **Privacy regulation.** The EU GDPR (Art. 17, "right to erasure") and India's Digital Personal Data Protection Act, 2023 give individuals the right to have their personal data erased. For a deployed model this means removing the person's influence on the model, not only on the database.
- **Membership privacy.** A model that memorised a sample leaks the fact that the sample was in the training set (membership inference attacks [8]). After a deletion request the model should not reveal membership any more.
- **Data quality and safety.** Mislabeled, poisoned, outdated or copyrighted data discovered after training must be removed without paying for a new training run.
- **Cost.** Retraining a large model for each request is too slow and too expensive; this is a systems problem as much as an ML problem.

## 1.3 Applications in vision

- **Face recognition / identification:** remove a person's identity (all their images) on request — the class-forgetting setting.
- **Medical imaging:** a patient withdraws consent; their scans must stop influencing a diagnostic model.
- **Autonomous driving and surveillance:** remove images captured without consent or images of specific locations/persons.
- **Content moderation and generative models:** remove harmful or copyrighted concepts; image classifiers are the standard test bed for these methods.

## 1.4 Forgetting scenarios and difficulty

Two settings are studied in the literature and in this project. In **class forgetting** an entire class is removed; the retrained model cannot recognise the class at all, so forget accuracy should drop to ≈0%. In **random / subset forgetting** individual samples are removed while other samples of the same classes stay; the retrained model still generalises to the forgotten samples (in our runs it classifies 89–92% of them correctly), so the target is *not* zero forget accuracy but the retrained model's behaviour. Unlearning is difficult because of three

competing goals: *forget* $D_f$, *retain* the utility on $D_r$ and the test set, and keep the *compute* small. An update that is too aggressive damages useful knowledge; one that is too gentle does not forget.

### 1.5 Approaches to unlearning

*Exact* unlearning guarantees equivalence with retraining, e.g. SISA [9] trains an ensemble on disjoint shards so that only one shard has to be retrained; it changes the training pipeline and still retrains. *Approximate* unlearning updates a single trained model: Fisher-information scrubbing [5], fine-tuning-based methods (gradient ascent, relabeling, distillation from the original model [1]), decision-boundary shifting [2] and saliency-masked updates [3]. Recent work studies *what makes unlearning hard* (memorisation and entanglement [4]), whether forgotten knowledge survives in the representations [7], and whether it re-appears for perturbed inputs [6]. This project covers the fine-tuning family, which is the practical choice for a deployed vision model, and these three evaluation questions.

### 1.6 Project objectives

1. Implement five unlearning methods (Gradient Ascent, GA + Descent, Teacher-Student, Random and Boundary Relabeling) and Gradient based Weight Saliency as an add-on to each, under one common data split, model and protocol.
2. Evaluate them beyond accuracy: membership inference, embedding entanglement, embedding separability (linear probing), robustness to perturbed inputs and to post-unlearning quantization, and cost (time, memory).
3. Diagnose failures and improve the methods where possible.

## 2 Experimental Setup

**Data.** CIFAR-10 (32×32 RGB, 10 classes). The 50k training images are split once into a 45k *train pool* and a 5k *validation* set (500 per class); the 10k test set is used only for reporting. Augmentation: random crop (padding 4) + horizontal flip.

**Model and training.** ResNet-18 (CIFAR variant, 11.2M parameters). The Original model is trained for 182 epochs with SGD (lr 0.1, cosine schedule, momentum 0.9, weight decay $5 \times 10^{-4}$, batch 256) and reaches **94.82%** test accuracy. Every Retrain reference uses the identical recipe on $D_r$.

**Scenarios.**

- **S1 (class forgetting):** $D_f$ = all 4,500 airplane images of the pool.
- **S2-10 / S2-30 / S2-50 (subset forgetting):** $D_f$ = a random 10/30/50% of each of the classes cat, dog and truck (1,350 / 4,050 / 6,750 images), different per seed.

For every scenario and seed (0, 1, 2) a Retrain reference is trained (12 models), plus 27 per-partition references for the entanglement study.

**Hyperparameter selection without test data.** Each method was first run in a pilot grid (seed 0) around the hyperparameters of its source paper, then improved where it failed (Section 6). Configurations are selected with a validation-only gap (`sel_gap`: forget, retain, validation accuracy and an MIA that uses validation images as negatives); the test set is never used for selection. The selected configuration is then run with 3 seeds.

**Systems aspects.** The whole dataset is kept on the GPU as `uint8` tensors with GPU-side augmentation (no data-loader bottleneck); training uses bf16 autocast and channels-last memory format, so a 182-epoch retraining takes 6–8 minutes on one A100. A project-wide GPU lock with out-of-memory retry lets long queues run unattended on a shared server; batches of up to 15 independent jobs were run in parallel over two GPUs (each $\leq$4 GB). Every run writes a record (configuration, per-epoch history, metrics, per-sample losses/predictions); the 1,845 records are aggregated automatically into the tables of this report (Table 1).

**Table 1:** Experiment scale: recorded runs per stage (each unlearning run takes 4–40 s, each retraining 6–8 min).

| Stage | Runs | Content |
|---|---|---|
| References | 70 | Original, 12 scenario Retrains + 27 partition Retrains, evaluations |
| Pilot grid | 161 | hyperparameters around each paper's setting (seed 0, S1 + S2-30) |
| Main grid | 120 | 5 methods $\times$ $\pm$WS $\times$ 4 scenarios $\times$ 3 seeds |
| Improvement stages | 409 | BN fixes, neighbour search, Teacher-Student schedules, confirmations (3 seeds) |
| Ablations | 110 | WS mask ratio 0.1–0.9 and random mask |
| Entanglement | 975 | Exp 1 (per partition) and Exp 2 (sequential + final evaluations) |
| Total | 1,845 | + perturbation, quantization and linear-probe evaluations of the checkpoints |

# 3 Unlearning Methods

Notation: $f_\theta$ the network, $\ell(\theta;x,y)$ the cross-entropy loss, $L(\theta;D)$ its mean over $D$, $\eta$ the learning rate. All methods start from $\theta_o$ and use SGD (momentum 0.9, weight decay $5\times10^{-4}$) with the standard augmentation. Table 2 lists the selected configurations.

## 3.1 M1: Gradient Ascent (GA)

The simplest baseline maximises the loss on the forget set:

$$\theta \leftarrow \theta + \eta\,\nabla_\theta L(\theta;D_f). \tag{1}$$

It uses only $D_f$ and has no term that protects the retain data, so it is prone to collateral damage and must be stopped early. Our configuration (G3) stops at iteration level (forget accuracy checked every 2 steps) and uses *mixed BatchNorm batches* (Section 6.1).

## 3.2 M2: Gradient Ascent + Descent (GA + GD)

Ascent on the forget set is balanced by descent on the retain set in the same update:

$$\min_\theta\ \beta\,L(\theta;D_r) - (1-\beta)\,L(\theta;D_f), \qquad \theta \leftarrow \theta - \eta\big(\beta\nabla L_r - (1-\beta)\nabla L_f\big). \tag{2}$$

$\beta$ controls the trade-off (higher $\beta$ = more retention). The method is very sensitive to $\beta$: we use $\beta = 0.99$ (S1) and 0.999 (S2), with gradient clipping at norm 1.

## 3.3 M3a: Teacher-Student unlearning

The frozen Original is the teacher $T$, a copy is the student $S$ [1]. Training alternates a *max-step* on the forget set, which pushes the student's output distribution away from the teacher, and a *min-step* on the retain set, which keeps it close and fits the labels:

$$\text{max-step:}\quad \min_\theta\ -\tfrac{1}{|D_f|}\textstyle\sum_{x\in D_f}\mathrm{KL}\big(p_T(x)\,\|\,p_S(x)\big), \tag{3}$$

$$\text{min-step:}\quad \min_\theta\ \tfrac{1}{|D_r|}\textstyle\sum_{(x,y)\in D_r}\alpha\,\mathrm{KL}\big(p_T(x)\,\|\,p_S(x)\big) + \gamma\,\ell(\theta;x,y), \tag{4}$$

with softmax temperature $\mathscr{T}$ in $p_T, p_S$. Max-steps are done only in the first few epochs (`msteps`), followed by extra min-steps.

### 3.4 M3b-R: Random Relabeling

Each forget image gets a random wrong label $y' \neq y$; the model is fine-tuned on the relabeled forget set together with the retain set [3]:

$$\min_{\theta} L(\theta; \{(x, y')\}_{D_f}) + \alpha L(\theta; D_r). \tag{5}$$

In each epoch we do one pass over the relabeled forget set and one pass over the retain set (as in the reference code).

### 3.5 M3b-B: Boundary Relabeling

Instead of a random label, each forget image is relabeled to its *nearest incorrect class*, so that the decision boundary moves only locally [2]. The paper finds the neighbour with one FGSM step on the frozen Original model:

$$x' = \mathrm{clip}_{[0,1]}\big(x + \varepsilon\, \mathrm{sign}(\nabla_x \ell(\theta_o; x, y))\big), \qquad y_{nb} = \arg\max_{c \neq y} f_{\theta_o}(x')_c, \tag{6}$$

and fine-tunes on $(x, y_{nb})$ using the forget set only (configuration C0, $\varepsilon = 0.1$). Our improved configuration **C1-mix** uses the Original's second-highest class on the clean image ($y_{nb} = \arg\max_{c \neq y} f_{\theta_o}(x)_c$, "logit2"), adds a retain pass and mixed BN batches (Section 6.3).

### 3.6 M4: Gradient based Weight Saliency (WS)

Weight Saliency [3] restricts the update to the weights that matter most for the forget set. The saliency of each parameter is the magnitude of the forget-loss gradient at the Original weights; the top fraction (ratio 0.5) is updated, the rest stays frozen:

$$g = \big|\nabla_\theta L(\theta; D_f)\big|_{\theta=\theta_o}, \qquad m_i = \mathbb{1}\big[g_i \geq \tau_{0.5}\big], \qquad \theta \leftarrow \theta - \eta\,(m \odot \nabla_\theta \mathscr{L}_{\text{method}}), \tag{7}$$

where $\tau_{0.5}$ is the global median of $g$. We apply the mask to all five methods; masked weights are reset to $\theta_o$ after each step (so momentum and weight decay cannot move them).

**Table 2:** Selected configurations (3 seeds). "Paper" = setting of the source paper used in the main grid; bold names are our improved configurations.

| Method | Scenario | Configuration |
|---|---|---|
| Gradient Ascent | S1 / S2 | **G3**: lr $3 \times 10^{-4}$ / $10^{-4}$, $\leq$10 epochs, batch 256, iteration-level early stop at Retrain forget acc, mixed BN batches (paper-range G0: lr $10^{-4}$, 5 epochs) |
| GA + Descent | S1 / S2 | D1: lr 0.01, $\beta$ 0.99 / 0.999, 5 epochs, clip 1.0 |
| Teacher-Student | S1 | T1: $\mathscr{T}$=4, $\alpha$=0.1, $\gamma$=1, 3 epochs (2 max-steps), forget/retain batch 128, lr $5 \times 10^{-4}$ |
| | S2 | **T4-best**: $\mathscr{T}$=4, $\alpha$=0.1, $\gamma$=0.1, 3 epochs (2 max-steps), forget batch 64 (S2-50: 128), retain batch 64 |
| Random Relabeling | S1 / S2 | R1: lr 0.005 / 0.02, 10 epochs, batch 256 |
| Boundary Relabeling | S1 | **C1-mix**: logit2 neighbour labels, lr $10^{-3}$, 10 epochs, retain pass, mixed BN (paper C0: FGSM $\varepsilon$=0.1, lr $10^{-5}$) |
| | S2 | C0 (no configuration improved on it in S2) |
| Weight Saliency | all | mask ratio 0.5 (50% of weights updated), added to every method |

## 4 Evaluation Metrics

### 4.1 Accuracy and distance to Retrain

**Forget accuracy** $\mathrm{Acc}_f$ (on $D_f$), **retain accuracy** $\mathrm{Acc}_r$ (on $D_r$) and **test accuracy** $\mathrm{Acc}_t$ are measured in eval mode without augmentation. For S1 we also report the test accuracy on the 9 remaining classes (*test other-cls*).

Since the goal is to behave like the retrained model, all metrics are summarised by the **average gap** to Retrain (Weight Saliency paper):

$$\text{avg_gap} = \tfrac{1}{4}\Big(|\Delta\text{Acc}_f| + |\Delta\text{Acc}_r| + |\Delta\text{Acc}_t| + |\Delta\text{MIA}|\Big), \qquad \Delta x = x(\theta_u) - x(\theta_r). \tag{8}$$

Lower is better; 0 means identical to Retrain on these four numbers. For the untouched Original it measures the gap that unlearning has to close.

## 4.2 Membership Inference Attack (MIA)

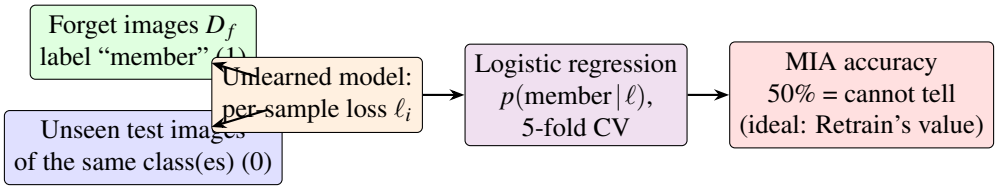



**Figure 2:** Loss-based membership inference attack used to measure privacy after unlearning.

After unlearning, an attacker should not be able to tell forget images from images the model never saw. For every forget image and every *same-class* test image we compute the loss $\ell_i = \ell(\theta_u; x_i, y_i)$ (eval mode, no augmentation). The two groups are balanced by sub-sampling ($n = \min(|D_f|, |\text{test}_{\text{same class}}|)$; 1,000 vs 1,000 in S1). An attacker $p(\text{member} \mid \ell) = \sigma(w\ell + b)$ (logistic regression) is trained and tested with stratified 5-fold cross-validation; the reported **MIA accuracy** is the mean held-out accuracy (Fig. 2). Using same-class negatives prevents the attacker from exploiting class differences. 50% means the attacker cannot separate the groups. The target is the Retrain value (49.6–51.0% in all scenarios); the Original scores 57.8% (S1) to 59.5% (S2). Values well *below* Retrain would also leak information (the "Streisand effect": forget samples recognisable by their unusually high loss). Figure 3 shows what the attacker sees: for the Original, forget images have systematically lower losses than unseen images (the long right tail of unseen images), which is the membership signal; after good unlearning the two distributions coincide.

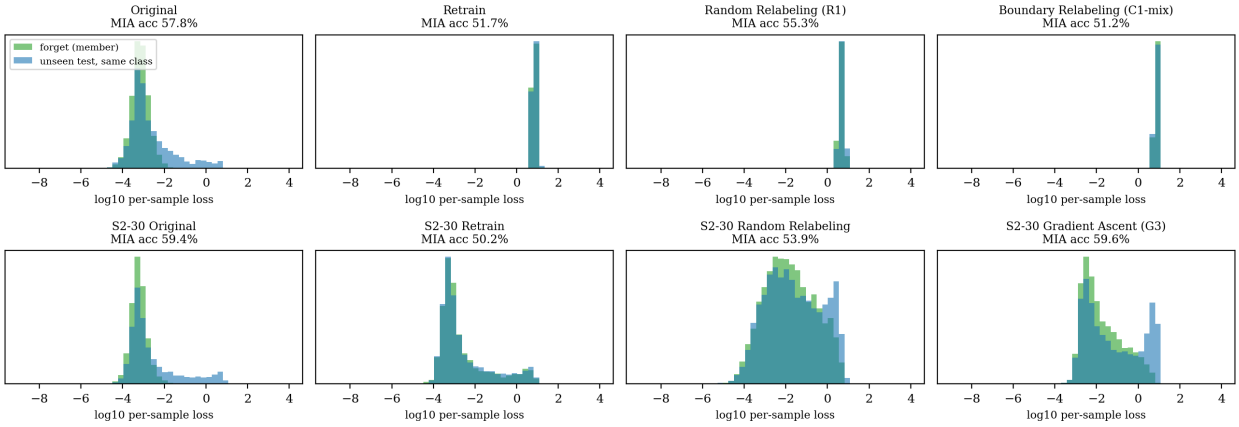



**Figure 3:** Per-sample loss distributions of forget images (green) and unseen same-class test images (blue), seed 0. Top: S1; bottom: S2-30. The more the distributions differ, the higher the MIA accuracy. In S1 the Original separates them by its low-loss forget images; in S2-30 Gradient Ascent leaves the forget images with lower losses than unseen images (MIA 59.6%).

## 4.3 Embedding entanglement

Some samples are harder to unlearn than others. We measure how strongly each forget image is *entangled* with the retain data in the Original model's 512-d penultimate feature space $\phi(\cdot)$:

$$e_i = \frac{1}{k} \sum_{j \in \mathcal{N}_k^R(i)} \big\|\phi(x_i) - \phi(x_j)\big\|_2, \qquad k = 50, \tag{9}$$

where $\mathcal{N}_k^R(i)$ are the $k$ nearest retain images. A small distance means the image looks like many retain images (**high** entanglement). The forget set is sorted by $e_i$ and split into three equal partitions — **low**, **medium**, **high** (S1: 1,500 images each; S2-10/30/50: 450 / 1,350 / 2,250; Fig. 4).

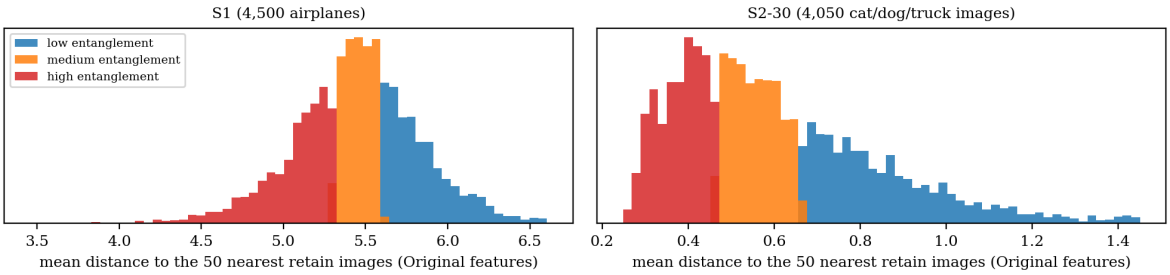



**Figure 4:** Distribution of the entanglement score $e_i$ over the forget set and the three partitions (seed 0).

- **Exp 1 (each partition alone).** Every method unlearns one partition starting from $\theta_o$. In *S1* the retain set is the normal S1 retain set (no airplanes), the other two partitions are not used, and the reference is the S1 Retrain evaluated on the partition. In *S2* the other partitions stay in the retain set and a Retrain is trained per partition. Hypothesis: forget accuracy after unlearning rises from low to high entanglement.
- **Exp 2 (sequential).** The three partitions are unlearned one after another (low $\rightarrow$ medium $\rightarrow$ high), each step starting from the previous model; the final model is compared with one-shot unlearning of the whole forget set.

### 4.4 Embedding separability: linear probe, NCC, k-NN purity

Low output accuracy does not prove that the forgotten class has left the representation [7]. For S1 we freeze the feature extractor $\phi_{\theta_u}$ and measure:

- **Linear probe:** a new linear classifier trained on frozen features of *all* 45k pool images (including the forgotten class); its accuracy on class-0 test images shows whether the class is linearly recoverable.
- **Nearest class centroid (NCC):** $\hat{y} = \arg\min_c \|\phi(x) - \mu_c\|_2$, with $\mu_c$ the mean feature of class $c$ (no training): does class 0 still form its own group?
- **10-NN purity:** share of the 10 nearest test neighbours of a class-0 test image that are also class 0 (cluster compactness).

### 4.5 Robustness: perturbed forget samples and quantization

**Perturbed forget samples** [6]: forgotten knowledge may reappear in the neighbourhood of a forget image. We evaluate $x' = \text{clip}(x + \tau\varepsilon)$, $\varepsilon \sim \mathcal{N}(0, I)$, with 20 draws per image and $\tau \in \{0.01, \ldots, 0.1\}$.

**Post-unlearning quantization.** Deployed models are often quantized. Unlearning updates are small, so rounding might map $\theta_u$ back onto the Original's grid and restore the forgotten knowledge (H1), or the added noise might increase forgetting (H2). We apply simulated weight-only, per-output-channel symmetric quantization to every Conv/Linear weight (BatchNorm in fp32):

$$s_c = \frac{\max_j |w_{c,j}|}{2^{b-1}-1}, \qquad \hat{w}_{c,j} = s_c \cdot \text{clip}\Big(\text{round}\big(w_{c,j}/s_c\big), -(2^{b-1}-1), 2^{b-1}-1\Big), \quad b \in \{8,6,4\}. \tag{10}$$

## 5 Results

All numbers are means over 3 seeds; the best value among the unlearning methods is highlighted.

## 5.1 Accuracy and MIA

**Table 3:** S1 (forget the airplane class). Accuracies and MIA in %, time = unlearning wall-clock time on one A100 (Retrain: training time). Paper-range main-grid results: Gradient Ascent G0 avg_gap 4.58, Boundary Relabeling C0 6.92.

| Model | WS | Forget | Retain | Test | Test other-cls | MIA | avg_gap | Time (s) |
|---|---|---|---|---|---|---|---|---|
| Retrain (reference) | | 0.00 | 100.00 | 85.38 | 94.86 | 51.02 | 0 | 401.6 |
| Original (no unlearning) | | 100.00 | 100.00 | 94.82 | 94.76 | 57.78 | 29.05 | – |
| Gradient Ascent (G3) | | 0.45 | 99.17 | 83.17 | 92.34 | 51.78 | 1.07 | **3.7** |
| | ✓ | 0.62 | 99.15 | 83.24 | 92.40 | 52.13 | 1.18 | 4.8 |
| GA + Descent (D1) | | **0.00** | 99.09 | 82.68 | 91.87 | 49.25 | 1.34 | 23.0 |
| | ✓ | **0.00** | 99.43 | 83.29 | 92.54 | 50.67 | 0.81 | 26.0 |
| Teacher-Student (T1) | | 0.33 | 99.99 | 84.89 | 94.30 | 52.58 | 0.60 | 20.7 |
| | ✓ | 2.56 | 99.99 | 85.02 | 94.26 | 52.18 | 1.02 | 25.8 |
| Random Relabeling (R1) | | **0.00** | **100.00** | 85.09 | 94.54 | 55.47 | 1.19 | 26.6 |
| | ✓ | **0.00** | **100.00** | 85.18 | **94.64** | 55.48 | 1.17 | 32.7 |
| Boundary Relabeling (C1-mix) | | **0.00** | **100.00** | 85.11 | 94.56 | **51.03** | **0.16** | 28.1 |
| | ✓ | **0.00** | **100.00** | 85.14 | 94.60 | 51.90 | 0.28 | 32.5 |

**Table 4:** S2 (forget a random 10/30/50% of cat, dog, truck). Forget/retain/test accuracy and MIA without WS; last column: avg_gap with WS.

| Scenario | Model | Forget | Retain | Test | MIA | avg_gap | avg_gap + WS |
|---|---|---|---|---|---|---|---|
| S2-10 | Retrain | 91.63 | 100.00 | 94.63 | 49.64 | 0 | |
| | Original | 100.00 | 100.00 | 94.82 | 58.95 | 4.47 | |
| | Gradient Ascent | 95.41 | 99.00 | 90.67 | 60.10 | 4.80 | 5.01 |
| | GA + Descent | 91.41 | 98.72 | 90.62 | 57.99 | **3.52** | 3.59 |
| | Teacher-Student | 97.48 | 99.55 | 93.07 | 57.02 | 3.81 | 3.87 |
| | Random Relabeling | 95.16 | 97.80 | 90.69 | 55.62 | 3.91 | 4.36 |
| | Boundary Relabeling | 73.56 | 92.49 | 84.94 | 59.25 | 11.22 | 11.31 |
| S2-30 | Retrain | 90.86 | 100.00 | 94.46 | 49.95 | 0 | |
| | Original | 100.00 | 100.00 | 94.82 | 59.46 | 4.75 | |
| | Gradient Ascent | 90.53 | 97.93 | 89.20 | 59.62 | 4.40 | 4.54 |
| | GA + Descent | 92.92 | 98.74 | 90.36 | 59.12 | 4.15 | 4.38 |
| | Teacher-Student | 94.60 | 98.85 | 92.17 | 56.13 | 3.34 | 3.47 |
| | Random Relabeling | 88.67 | 96.51 | 90.34 | 53.16 | 3.25 | **3.10** |
| | Boundary Relabeling | 75.54 | 94.33 | 85.32 | 59.52 | 9.92 | 10.12 |
| S2-50 | Retrain | 88.74 | 100.00 | 93.75 | 50.75 | 0 | |
| | Original | 100.00 | 100.00 | 94.82 | 59.47 | 5.26 | |
| | Gradient Ascent | 88.32 | 98.01 | 88.64 | 59.46 | 4.18 | 4.18 |
| | GA + Descent | 89.26 | 98.44 | 89.26 | 59.07 | 3.80 | 4.04 |
| | Teacher-Student | 94.08 | 99.06 | 91.86 | 57.04 | 3.62 | 3.43 |
| | Random Relabeling | 81.07 | 93.31 | 87.42 | 51.78 | 5.43 | **2.58** |
| | Boundary Relabeling | 75.11 | 95.59 | 85.05 | 59.92 | 8.98 | 8.76 |

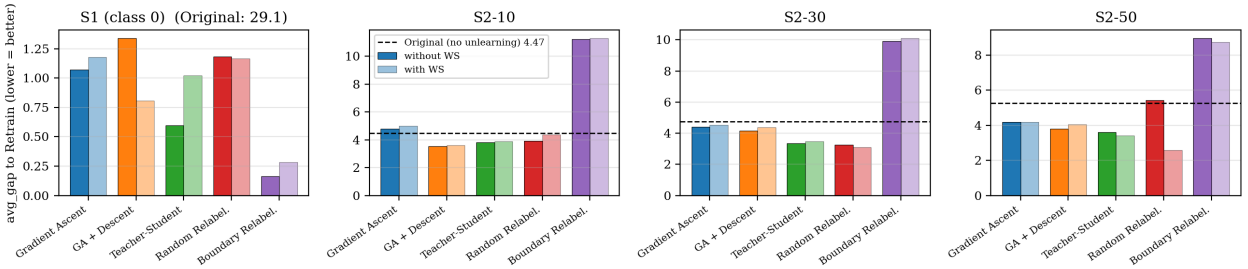



**Figure 5:** avg_gap to Retrain of the best configuration of every method (solid: without WS, light: with WS); dashed line: doing nothing (Original).

**Key observations.** (1) **Class forgetting is solved at the output level:** every method removes the airplane class (forget acc $\leq$2.6%) while keeping retain accuracy $\geq$99%; our Boundary C1-mix matches Retrain almost exactly (avg_gap 0.16, MIA 51.03 vs 51.02) at **14$\times$ lower cost** than retraining (Gradient Ascent: 108$\times$). (2) **The two forget-only methods were the weakest in their paper settings** (GA 4.58, Boundary 6.92) and became competitive only after the BatchNorm fix (1.07, 0.16; Section 6.1). (3) **Subset forgetting is much harder:** the best methods only shrink the gap of doing nothing from 4.5–5.3 to 2.6–3.5. Retrain still classifies 89–92% of the forget images correctly, so the target is to remove the *memorisation* signal, not the class: the MIA stays at 52–60% (Retrain $\approx$50%). Random Relabeling is the closest on MIA (51.8–55.6%); Boundary Relabeling has no good S2 setting (it lowers forget accuracy far below Retrain and damages same-class retain images). (4) **MIA reveals the "Streisand effect" of random labels:** in S1 Random Relabeling reaches 0% forget accuracy but 55.5% MIA — its forget images get recognisably high losses.

## 5.2 Embedding separability (linear probe)

**Table 5:** S1: is the forgotten class still present in the features? Accuracy on class-0 test images (%) of the model output, a fresh linear probe and the nearest class centroid (NCC), and the 10-NN purity of class 0 (best configurations without WS; WS changes each value by at most 4.5 points).

| Model | Output acc (class 0) | Linear probe (class 0) | NCC (class 0) | 10-NN purity |
|---|---|---|---|---|
| Original | 95.40 | 95.50 | 95.40 | 93.00 |
| Retrain | 0.00 | 87.73 | 41.43 | 52.58 |
| Gradient Ascent (G3) | 0.60 | 86.20 | 30.33 | 32.63 |
| GA + Descent (D1) | 0.00 | 81.80 | 32.47 | 32.38 |
| Teacher-Student (T1) | 0.17 | 88.50 | 45.60 | 56.07 |
| Random Relabeling (R1) | 0.03 | 94.30 | 82.10 | 89.24 |
| Boundary Relabeling (C1-mix) | 0.00 | 94.33 | 74.77 | 82.16 |

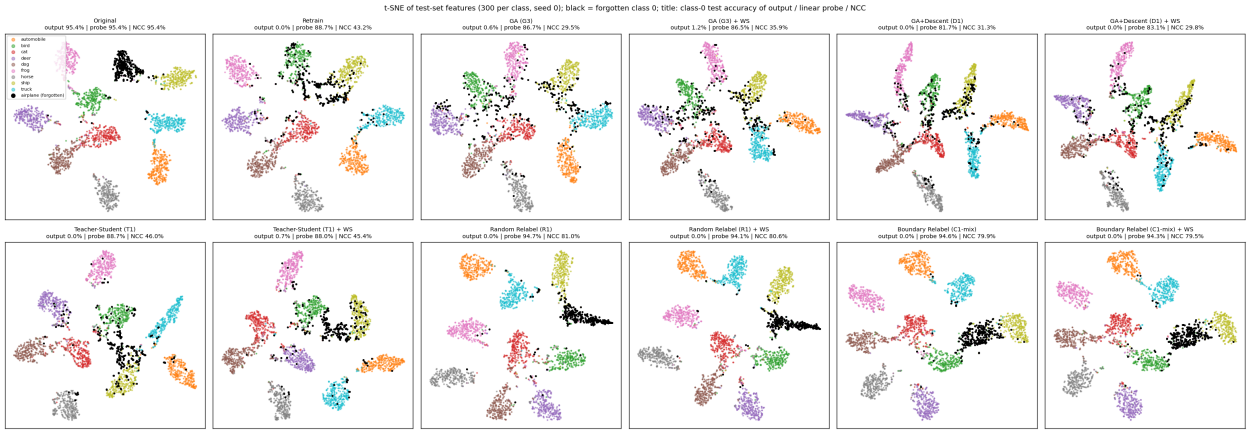



**Figure 6:** t-SNE of test-set features (300 images per class, seed 0); black = forgotten class (airplane). Panel titles: class-0 accuracy of output / linear probe / NCC.

**Observations.** (1) A linear probe recovers the forgotten class at 82–94% for *every* method — but also at 87.7% for Retrain, which never saw an airplane: features learned from the other nine classes still describe airplanes, so a high probe accuracy alone is not proof of residual memory. (2) The cluster measures separate the methods into two groups. **GA, GA + Descent and Teacher-Student disperse the airplane cluster** to Retrain level or beyond (NCC 30–46%, purity 32–56%; Retrain 41% / 53%). **Random and Boundary Relabeling leave it intact** (NCC 75–82%, purity 82–89%; Fig. 6, bottom right): their forgetting happens in the classifier head, as reported in the "Illusion of Unlearning" paper [7]. (3) Boundary C1-mix is therefore the best method on output metrics but not on representation metrics; Teacher-Student is the closest to Retrain at the representation level and second-best at the output level (0.60).

## 5.3 Entanglement and unlearning difficulty

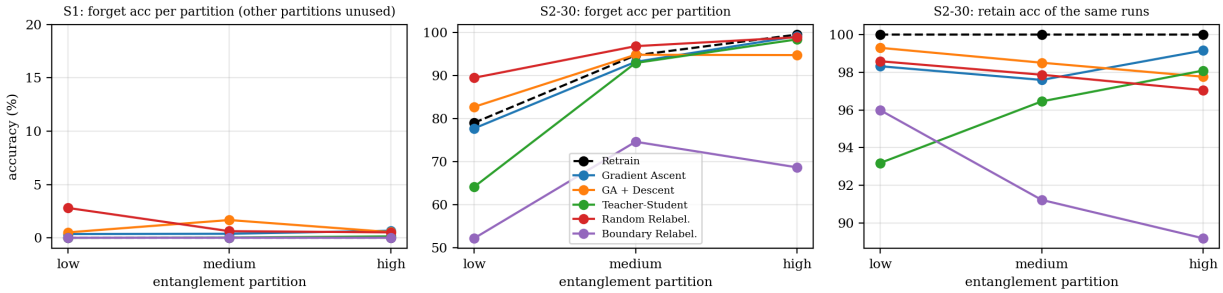



**Figure 7:** Exp 1: forget accuracy after unlearning each entanglement partition alone (S1 left, S2-30 middle) and the retain accuracy of the S2-30 runs (right); dashed: Retrain.

**Table 6:** Exp 1, S1 (class forgetting; forget = one partition, other two partitions unused). Forget accuracy on the partition, accuracy on the 3,000 unused airplanes, and avg_gap (%).

| | Forget acc | | | Unused airplanes acc | | | avg_gap | | |
|---|---|---|---|---|---|---|---|---|---|
| Model | low | med | high | low | med | high | low | med | high |
| Original | 100 | 100 | 100 | 100 | 100 | 100 | 28.59 | 29.52 | 29.27 |
| Gradient Ascent | 0.4 | 0.4 | 0.7 | 0.2 | 0.4 | 5.4 | 1.50 | 1.30 | 0.84 |
| GA + Descent | 0.5 | 1.7 | 0.5 | 0.1 | 0.4 | 0.8 | 1.39 | 2.56 | 3.46 |
| Teacher-Student | 0.0 | 0.0 | 0.1 | 0.0 | 0.0 | 0.8 | 0.56 | 1.39 | 0.84 |
| Random Relabeling | 2.8 | 0.6 | 0.5 | 0.2 | 1.4 | 2.2 | 2.06 | 0.53 | 1.36 |
| Boundary Relabeling | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.74 | 0.48 | 0.90 |

**Table 7:** Exp 1, S2-30 (subset forgetting). Retrain forget accuracy per partition: 79.0 / 94.7 / 99.6%.

| | Forget acc | | | Retain acc | | | avg_gap | | |
|---|---|---|---|---|---|---|---|---|---|
| Model | low | med | high | low | med | high | low | med | high |
| Original | 100 | 100 | 100 | 100 | 100 | 100 | 5.85 | 3.06 | 0.75 |
| Gradient Ascent | 77.7 | 93.2 | 99.2 | 98.3 | 97.6 | 99.2 | 3.37 | 4.42 | 1.91 |
| GA + Descent | 82.7 | 94.9 | 94.8 | 99.3 | 98.5 | 97.8 | 3.12 | 3.58 | 3.88 |
| Teacher-Student | 64.1 | 93.0 | 98.4 | 93.2 | 96.5 | 98.1 | 7.62 | 3.73 | 1.96 |
| Random Relabeling | 89.5 | 96.9 | 98.9 | 98.6 | 97.9 | 97.1 | 5.49 | 3.26 | 3.33 |
| Boundary Relabeling | 52.1 | 74.6 | 68.7 | 96.0 | 91.2 | 89.2 | 11.78 | 12.17 | 14.65 |

**Table 8:** Exp 2: avg_gap of one-shot unlearning vs sequential low → medium → high unlearning (without WS; better of the pair highlighted).

| | S1 | | S2-10 | | S2-30 | | S2-50 | |
|---|---|---|---|---|---|---|---|---|
| Method | one-shot | seq. | one-shot | seq. | one-shot | seq. | one-shot | seq. |
| Gradient Ascent | **1.07** | 1.34 | **4.80** | 5.07 | **4.40** | 7.37 | **4.18** | 6.53 |
| GA + Descent | **1.34** | 3.04 | **3.52** | 4.89 | **4.15** | 6.83 | **3.80** | 9.27 |
| Teacher-Student | **0.60** | 30.43 | 3.81 | **2.78** | **3.34** | 63.87 | **3.62** | 61.64 |
| Random Relabeling | **1.19** | 28.65 | 3.91 | **2.98** | 3.25 | **2.67** | 5.43 | **2.55** |
| Boundary Relabeling | **0.16** | 27.91 | **11.22** | 13.51 | **9.92** | 14.95 | **8.98** | 13.33 |

**Observations.** (1) **Subset forgetting: highly entangled samples are harder to forget.** For every method the forget accuracy after unlearning rises from the low to the high partition (S2-30: Gradient Ascent 77.7 → 99.2%, Teacher-Student 64.1 → 98.4%). Methods that push the entangled partitions down pay with retain accuracy (Boundary: 89.2% retain on the high run), confirming the hypothesis that entangled samples cannot be removed without hurting the retain data. Retrain shows the same trend (79.0 → 99.6%): low-entanglement images are the *memorised* ones. (2) **Class forgetting: entanglement does not make forgetting harder.** Every partition is forgotten almost completely (≤2.8%) and the forgetting spreads to the unused airplanes (0–5.4%): a class is one shared concept, removed through any third of its images. Entanglement shows only weakly: after the high run more of the unused airplanes survive (GA 5.4% vs 0.2%), and GA + Descent's MIA rises on the high partition (avg_gap 1.39 → 3.46). (3) **Sequential unlearning helps only Random Relabeling** in every S2 ratio (S2-50: 5.43 → 2.55). Ascent-based methods accumulate damage over the steps; Teacher-Student collapses when the max-steps add up (S2-30/50, S1 without WS). In S1, methods with a retain pass re-learn airplanes from the partitions still in the retain set during steps 1–2 (final forget acc 96–98%).

## 5.4 Effect of Gradient based Weight Saliency

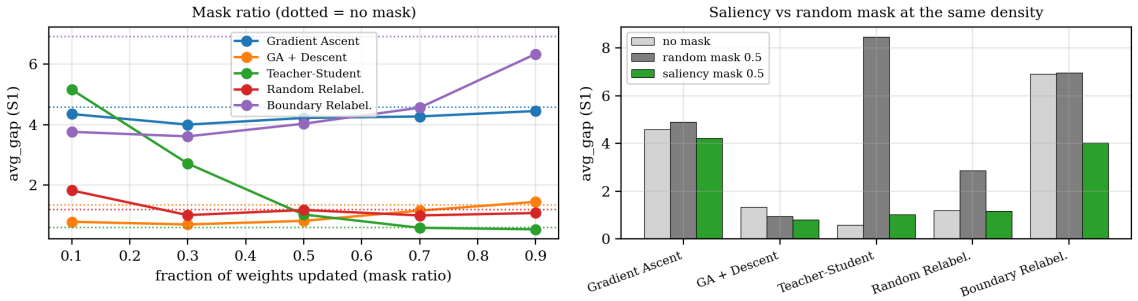



**Figure 8:** Weight Saliency ablation on the main-grid configurations (S1, seeds 0–1). Left: avg_gap vs mask ratio (dotted: no mask). Right: saliency mask vs a random mask of the same density.

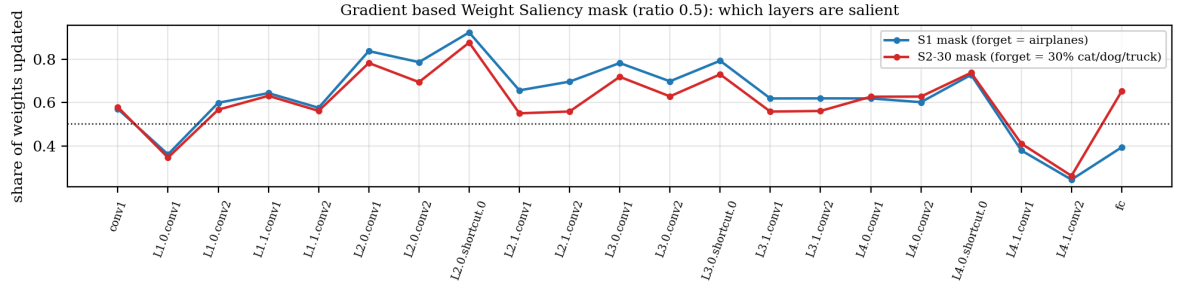



**Figure 9:** Share of weights selected by the saliency mask in each convolution / linear layer (global threshold, ratio 0.5; seed 0).

The global threshold does not select weights uniformly (Fig. 9): the middle of the network (layer2–layer3 and the shortcut convolutions, up to 92% of their weights) carries the largest forget-loss gradients, the last

residual block the smallest (25–41%). The S1 and S2-30 masks are very similar except for the classifier layer (39% vs 65%). On the paper-range (main-grid) configurations the mask improves the methods that damage the retain set: Boundary Relabeling S1 6.92 → 4.03, GA + Descent S1 1.34 → 0.81, and Random Relabeling S2-50 5.43 → 2.58 (the Weight Saliency paper's own method). It gives no gain where the base method is already close to Retrain (Teacher-Student S1 0.60 → 1.02) or barely changes the model (S2 Gradient Ascent / Boundary).

**Observations.** (1) **Saliency, not sparsity, matters**: at the same density a random mask is worse than the saliency mask for every method (Teacher-Student 8.47 vs 1.02, Random Relabeling 2.88 vs 1.17). (2) The best ratio is method-dependent: 0.3 for GA, GA + Descent and Boundary; ≥0.7 for Teacher-Student and Random Relabeling. (3) WS mainly adds *stability*: it prevents the sequential Teacher-Student collapse in S1 (30.43 → 0.98) and the over-aggressive Random Relabeling on S2-50. (4) It does not change what the features encode (probe/NCC within 4.5 points), costs 1.7–2.4 s for the mask and raises peak GPU memory from 1.1 to 3.8 GB (copy of $\theta_o$ + mask). (5) It cannot prevent the BatchNorm artefact: BN running statistics are buffers, not masked weights.

## 5.5 Robustness to perturbed forget samples

**Table 9:** Accuracy on Gaussian-perturbed forget images ($\tau = 0.03$, 20 draws per image) and recovery (= % of forget images wrong when clean but correct under ≥1 perturbation); main-grid models, without WS.

| | S1 | | S2-30 | | |
|---|---|---|---|---|---|
| Model | clean forget | perturbed | clean forget | perturbed | recovery |
| Original | 100.0 | 80.6 | 100.0 | 91.6 | 0.0 |
| Retrain | 0.0 | 0.0 | 90.9 | 78.4 | 44.5 |
| Gradient Ascent | 5.3 | 0.0 | 87.5 | 45.5 | 21.7 |
| GA + Descent | 0.0 | 0.0 | 92.9 | 56.1 | 31.3 |
| Teacher-Student | 0.3 | 0.0 | 99.6 | 87.5 | 71.6 |
| Random Relabeling | 0.0 | 0.0 | 88.7 | 71.0 | 41.4 |
| Boundary Relabeling | 23.6 | 0.8 | 75.5 | 33.0 | 20.2 |

In S1 no method shows residual knowledge under noise. In S2-30 Retrain itself recognises 78.4% of the perturbed forget images, so 0 is not the target: Teacher-Student retains *more* than Retrain (87.5%, residual knowledge), Gradient Ascent and Boundary Relabeling *less* (over-unlearning), and Random Relabeling is the closest (71.0%).

## 5.6 Post-unlearning quantization

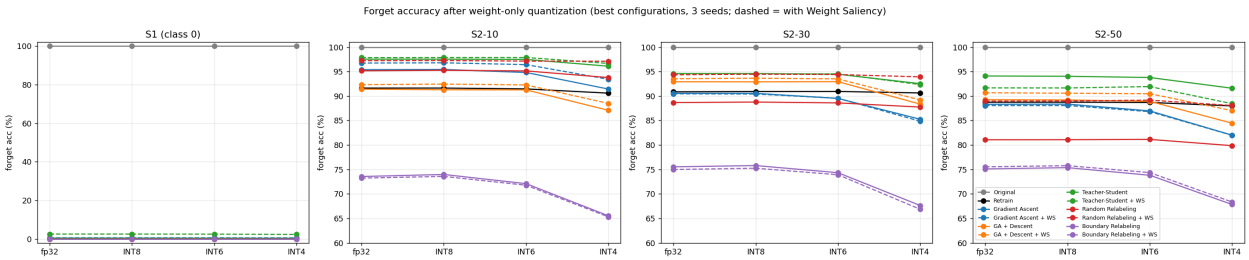
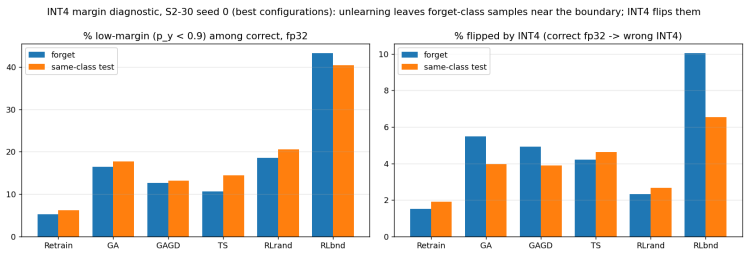


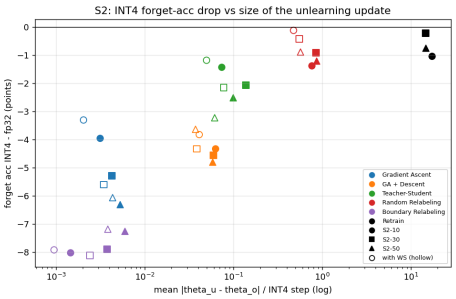

**Figure 10:** Forget accuracy after weight-only quantization of the best configurations (dashed: with WS).

**Table 10:** Change caused by INT4 quantization (INT4 − fp32, percentage points; best configurations without WS). Last column: mean $|\theta_u - \theta_o|$ in units of the INT4 step (S2-30).

| | S1 | Δ forget acc, S2 | | | S2-30 | | S2-30 |
|---|---|---|---|---|---|---|---|
| Model | Δ forget | S2-10 | S2-30 | S2-50 | Δ retain | Δ test | $\lvert\Delta\theta\rvert$/step |
| Retrain | 0.00 | −1.04 | −0.23 | −0.75 | 0.00 | −0.33 | 14.63 |
| Gradient Ascent | −0.07 | −3.95 | −5.29 | −6.31 | −1.17 | −1.28 | 0.004 |
| GA + Descent | 0.00 | −4.32 | −4.57 | −4.80 | −0.97 | −1.19 | 0.060 |
| Teacher-Student | −0.06 | −1.43 | −2.07 | −2.50 | −0.59 | −0.81 | 0.137 |
| Random Relabeling | +0.01 | −1.38 | −0.92 | −1.20 | −0.63 | −0.67 | 0.850 |
| Boundary Relabeling | 0.00 | −8.02 | −7.91 | −7.26 | −1.84 | −1.67 | 0.004 |




**(a)** Margin diagnostic (S2-30, seed 0)




**(b)** INT4 forget drop vs update size (S2)

**Figure 11:** Why INT4 lowers S2 forget accuracy. (a) Left: share of correctly classified images with true-class probability $p_y < 0.9$; right: share flipped from correct (fp32) to wrong (INT4). (b) INT4 forget-accuracy drop against the mean weight change in units of the INT4 step (hollow: with WS).

**Observations.** (1) **H1 rejected: quantization never restores forgotten knowledge** — for no method, scenario or bit width (largest forget-accuracy increase +0.42 at INT8); S1 forgetting is stable even at INT4. INT8 and INT6 change forget accuracy by at most 1.5 points. (2) **H2 only as non-selective degradation:** in S2, INT4 lowers forget accuracy for every method, but retain and test accuracy drop too and same-class test images flip at a similar rate as forget images. (3) **Why:** unlearning leaves many forget-class images correct but with a low margin (16.5% for GA and 43.3% for Boundary vs 5.3% for Retrain); INT4 rounding acts as small weight noise and flips them (Fig. 11). The drop is largest for the methods with the *smallest* weight change (GA, Boundary C0: ≈0.004 INT4 steps) and smallest for Random Relabeling (0.85 steps). (4) INT4 brings a model closer to Retrain only if it was under-forgetting (Teacher-Student: avg_gap 3.33 → 3.07 on S2-30); INT8 is safe for every unlearned model.

## 5.7 Computational cost

**Table 11:** Cost of the best configurations in S1 on one A100 (wall-clock unlearning time without evaluation; peak allocated GPU memory). Retraining from scratch: 401.6 s, 1.06 GB.

| Method | Time (s) | Speed-up | Peak mem. (GB) | Time + WS (s) | Mem. + WS (GB) |
|---|---|---|---|---|---|
| Gradient Ascent (G3) | 3.7 | 108× | 1.83 | 4.8 | 3.83 |
| GA + Descent (D1) | 23.0 | 17× | 1.70 | 26.0 | 3.83 |
| Teacher-Student (T1) | 20.7 | 19× | 1.10 | 25.8 | 3.83 |
| Random Relabeling (R1) | 26.6 | 15× | 1.10 | 32.7 | 3.83 |
| Boundary Relabeling (C1-mix) | 28.1 | 14× | 1.83 | 32.5 | 3.83 |

All methods are 14–108× faster than retraining. The methods with a retain pass (GA + Descent, Teacher-Student, the relabelings) are dominated by the retain-set iterations; Gradient Ascent touches only the forget set and stops after 16–50 iterations. Mixed BN batches (GA, Boundary) double the batch and raise memory to 1.8 GB; the WS mask adds ≈2 s and a copy of the weights (3.8 GB).

# 6 Method-specific Observations and Improvements

## 6.1 The BatchNorm-statistics problem (Gradient Ascent, Boundary Relabeling, GA + Descent)

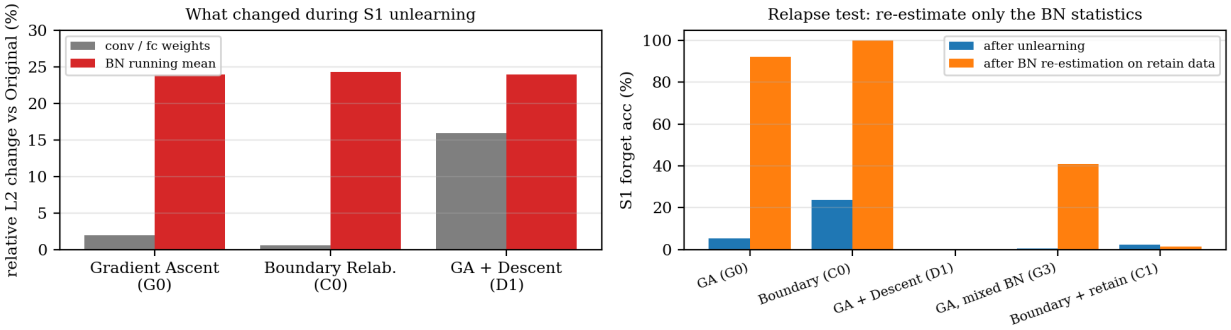



**Figure 12:** S1 BatchNorm artefact. Left: relative change of the weights vs the BN running means during unlearning (paper-range configurations). Right: forget accuracy before and after re-estimating only the BN statistics on retain data.

Forget-only methods train in BN train mode on batches that contain *only* forget images (in S1: only airplanes). Every forward pass updates the stored statistics, $\mu_{\text{run}} \leftarrow 0.9\,\mu_{\text{run}} + 0.1\,\mu_{\text{batch}}$, so after ≈30 steps ($0.9^{30} \approx 4\%$) the running mean/variance are airplane statistics. At test time all images are then normalised with airplane statistics, which suppresses airplane-specific activations: **the model "forgets" without its weights changing**. Evidence (Fig. 12): with paper settings the weights of Gradient Ascent change by 2.0% (Boundary: 0.57%) but the BN running means by ≈24%; re-estimating only the BN statistics on retain data restores forget accuracy from 5.3% to 92% (GA) and from 23.6% to 99.9% (Boundary). None of the source papers mentions this; in their random-forgetting setting the forget batches contain all classes, so BN barely drifts.

**Fix: mixed BN batches.** A retain batch is concatenated to every forget forward pass, so BN always sees representative statistics, while the loss is still computed on the forget part only (the method itself is unchanged). This moved Gradient Ascent from 4.58 to **1.07** and Boundary Relabeling (with a retain pass and logit2 labels) to **0.16**. Freezing BN instead made Gradient Ascent explode (retain 22–76%) because nothing bounds the unbounded ascent loss any more; it needed gradient clipping (retain 92.5%). GA + Descent's repository implementation also forwards forget batches separately (BN mean change 23.9%), but its retain loss changes the weights substantially (15.9%), so its forgetting survives BN re-estimation (0% → 0%).

## 6.2 Teacher-Student: soft-label re-teaching and collapse

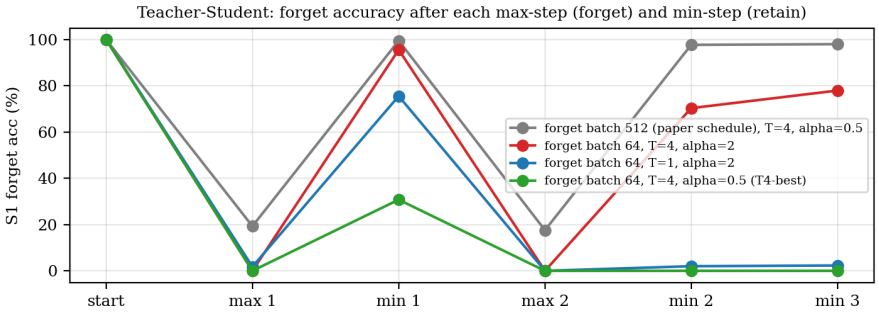



**Figure 13:** Teacher-Student in S1 (seed 0): forget accuracy right after each max-step and each min-step. Every max-step removes the class, the min-steps on the retain set (which contains no airplanes) bring it back unless the soft-label term is weak or the temperature is 1.

With the paper's settings (forget batch 512, $\alpha = \gamma = 1$) the student did not forget (forget acc 98–100%, avg_gap 28.57). Per-epoch logging (Fig. 13) showed forget accuracy near 0% right after each max-step and

back to ≈99% after the min-step, although the S1 retain set contains no airplanes. Two causes: (i) **soft-label re-teaching** — at $\mathscr{T} = 4$ the teacher gives the airplane class a mean probability of 0.050 on retain images (0.0001 at $\mathscr{T} = 1$) and it is the second class for 44.7% of ships and 24.9% of birds, so matching the soft labels rebuilds an airplane output on airplane-like features; (ii) **too few max-step updates** — with batch 512 an epoch has 9 forget updates against ≈316 retain updates. A small KL weight ($\alpha = 0.1$) with forget batch 128 fixed S1 (avg_gap 0.60). In S2 the retain set contains the same classes and the method is *bistable*: too few max-step updates forget nothing, too many collapse the model (retain 12–34%). What matters is the number of forget updates ($|D_f|$/batch × msteps); scaling the forget batch with the forget-set size (64 for S2-10/30, 128 for S2-50) made it stable on all seeds (S2-50: 58.2 → 3.62).

### 6.3 Boundary Relabeling: finding the nearest class

In the Original's feature space the nearest retain class of 84% of the airplanes is *ship*, but one FGSM step with $\varepsilon = 0.1$ assigns bird to 2,008, frog to 1,723 and ship to only 288 images: the large step lands in noise-like regions rather than at the nearest boundary. Using the second-highest class of the clean image (logit2: bird 1,771, ship 1,666, truck 366) or an iterative search that takes the first class crossed gives consistent labels; only with consistent labels does an added retain pass help (forget 13% with FGSM labels vs 0–2% with logit2). After unlearning, however, the forget images still form their own cluster next to bird/ship (probe 94%): the cross-entropy to the neighbour label is satisfied as soon as the logits cross the boundary, so the head changes and the backbone barely does.

### 6.4 Other issues found and fixed

- **GA overshoot:** with epoch-level stopping Gradient Ascent passes 0% forget accuracy early in the epoch and keeps damaging the retain set; iteration-level early stopping (every 2 steps on a fixed 1,000-image forget subset) stops it after 16–50 iterations.
- **GA + Descent's $\beta$ is very sensitive:** $\beta$ = 0.95 at lr 0.03 collapses (retain 25%), the paper's large-scale 0.9999 under-forgets (62%).
- **Divergence masked as success:** NaN logits make `argmax` return class 0 and report 100% forget accuracy; a `diverged` flag and a NaN-safe MIA exclude such runs.
- **Random Relabeling** chosen on S2-30 was too aggressive on S2-50 (retain 93.3%); the WS mask fixes it (avg_gap 5.43 → 2.58).

## 7 Overall Comparison of the Methods

**Table 12:** Summary of all evaluations (best configurations). avg_gap: S1 / S2-30; MIA: S1 (Retrain 51.0%); cluster: NCC accuracy on class 0 (Retrain 41%); INT4: forget-accuracy change on S2-30; time: S1.

| Method | S1 gap | S2-30 gap | MIA | Cluster | INT4 | Time | Main strength / weakness |
|---|---|---|---|---|---|---|---|
| Gradient Ascent | 1.07 | 4.40 | 51.8 | 30% | −5.3 | 3.7 s | fastest; needs mixed BN + iteration-level stop; over-forgets in S2 |
| GA + Descent | 1.34 | 4.15 | 49.3 | 32% | −4.6 | 23 s | robust forgetting (retain loss); very sensitive to $\beta$ |
| Teacher-Student | 0.60 | 3.34 | 52.6 | 46% | −2.1 | 21 s | closest to Retrain at feature level, second-best S1 output; collapses if max-steps add up |
| Random Relabeling | 1.19 | 3.25 | 55.5 | 82% | −0.9 | 27 s | best in S2 (with WS), most quantization-stable; Streisand effect in S1; head-only forgetting |
| Boundary Relabeling | **0.16** | 9.92 | **51.0** | 75% | −7.9 | 28 s | best S1 after our fixes (C1-mix); no good S2 setting; head-only forgetting |

# 8 Discussion: why subset forgetting looks "unsuccessful"

In subset forgetting no method reaches a low forget accuracy without hurting the retain data. This is the expected behaviour, not a failure: the retrained model keeps 88.7–91.6% forget accuracy because the remaining images of the same classes teach the same concepts. The Weight Saliency paper [3] reports the same for random forgetting of 10% and 50% of CIFAR-10 (its "unlearning accuracy" UA = 100−forget accuracy): Retrain forget accuracy 94.76% / 92.09% and their best method 97.15% / 92.25%, in line with our Retrain 91.63% (S2-10) / 88.74% (S2-50) and our best methods. A forget accuracy far below Retrain would make the forget images *more* identifiable (higher loss than unseen data). The real gap in subset forgetting is privacy: the MIA of all methods stays at 52–60% against 50% for Retrain, driven by the memorised, low-entanglement samples (Retrain recognises only 79% of them). Targeting these samples and matching the forget-loss distribution to that of unseen data are the natural next steps.

# 9 Conclusion

We implemented five unlearning methods and Gradient based Weight Saliency for ResNet-18 on CIFAR-10 under one protocol with 3 seeds and a retrained reference for every scenario, and evaluated them with accuracy, membership inference, entanglement, embedding separability, perturbation, quantization and cost (1,845 recorded runs).

- **Class forgetting** is solved at the output level by all methods at 14–108$\times$ lower cost than retraining. Our improved Boundary Relabeling (C1-mix) matches Retrain on accuracy and MIA (avg_gap 0.16 vs 6.92 for the paper setting), Teacher-Student is the closest to Retrain at the representation level and second-best at the output level.
- **A BatchNorm-statistics artefact** makes forget-only methods look successful while the weights barely change; mixed BN batches remove it. Evaluations should include a BN re-estimation check.
- **Output-level forgetting is not representation-level forgetting:** the relabeling methods leave the forgotten class as an intact feature cluster, while ascent-based methods and Teacher-Student disperse it.
- **Subset forgetting** is limited by generalisation: Retrain keeps $\approx$90% forget accuracy; highly entangled samples are hardest to forget and the remaining gap is in privacy (MIA). Random Relabeling + WS is the best method there, and the only one helped by sequential (low $\rightarrow$ high) unlearning.
- **Robustness:** weight quantization (INT8/6/4) never restores forgotten knowledge; INT4 degrades low-margin forget-class samples non-selectively.

**Future work:** feature-level losses that merge forget features into neighbouring classes, memorisation-targeted unlearning for subset forgetting, stronger per-example attacks (LiRA), and CIFAR-100 / larger models.

# References

[1] M. Kurmanji, P. Triantafillou, J. Hayes, E. Triantafillou, "Towards Unbounded Machine Unlearning," *NeurIPS*, 2023.

[2] M. Chen, W. Gao, G. Liu, K. Peng, C. Wang, "Boundary Unlearning: Rapid Forgetting of Deep Networks via Shifting the Decision Boundary," *CVPR*, 2023.

[3] C. Fan, J. Liu, Y. Zhang, E. Wong, D. Wei, S. Liu, "SalUn: Empowering Machine Unlearning via Gradient-based Weight Saliency in both Image Classification and Generation," *ICLR*, 2024.

[4] K. Zhao, M. Kurmanji, G.-O. Bărbulescu, E. Triantafillou, P. Triantafillou, "What Makes Unlearning Hard and What to Do About It," *NeurIPS*, 2024.

[5] A. Golatkar, A. Achille, S. Soatto, "Eternal Sunshine of the Spotless Net: Selective Forgetting in Deep Networks," *CVPR*, 2020.

[6] "The Unseen Threat: Residual Knowledge in Machine Unlearning under Perturbed Samples," *NeurIPS*, 2025.

[7] "An Illusion of Unlearning? Assessing Machine Unlearning Through Internal Representations," *PMLR*, 2026.

[8] R. Shokri, M. Stronati, C. Song, V. Shmatikov, "Membership Inference Attacks against Machine Learning Models," *IEEE S&P*, 2017.

[9] L. Bourtoule et al., "Machine Unlearning," *IEEE S&P*, 2021.

[10] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[11] S. Ioffe, C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *ICML*, 2015.

[12] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Tech. report, University of Toronto, 2009.